

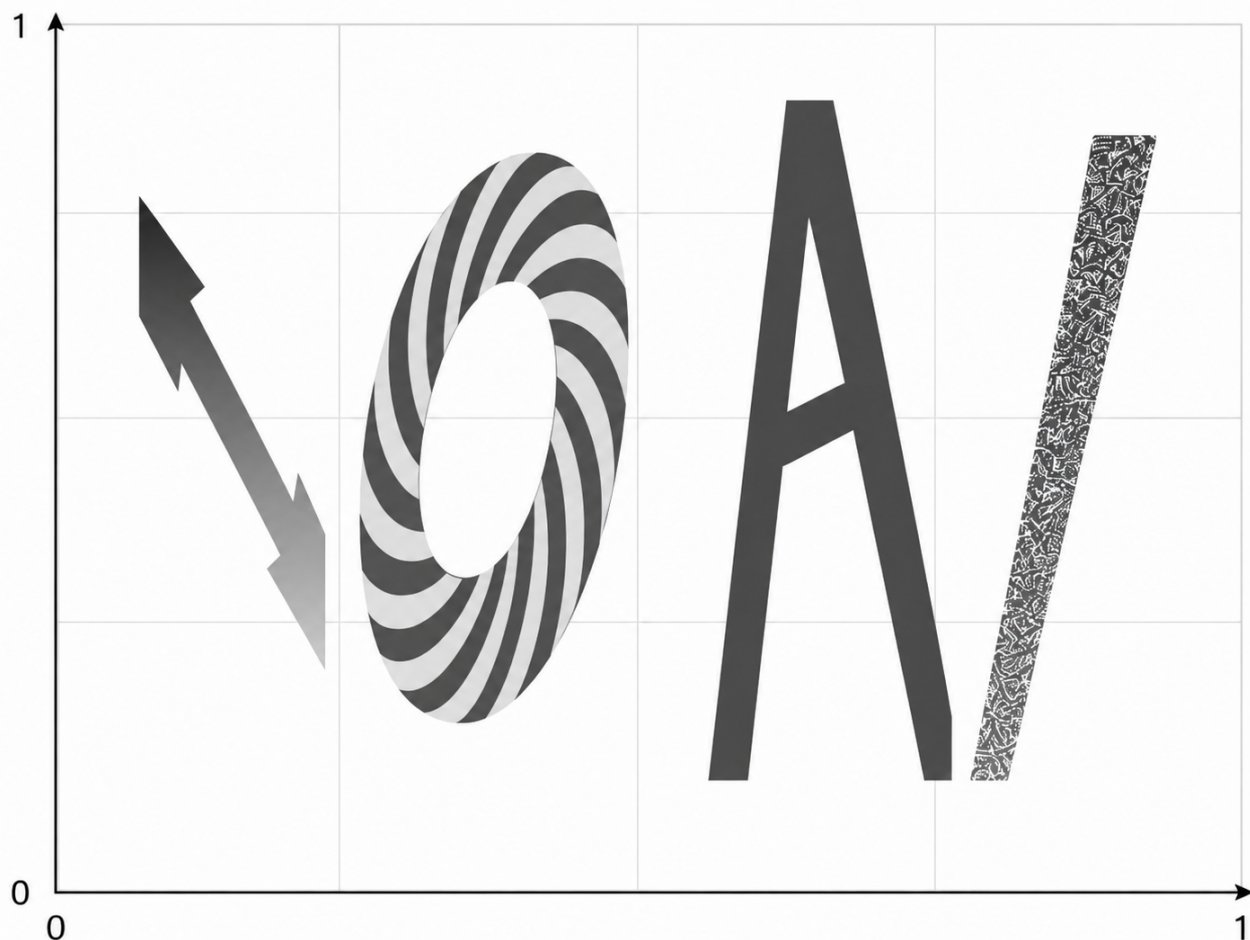
Pole IOAI

- **Limit czasu:** 5 minut
- **Pamięć dyskowa:** 5 GB
- **Rozmiar rozwiązania:** `solution.ipynb`, `custom_model.py` ≤ 1 MB łącznie
- **Modele wstępnie wytrenowane:** brak — tylko trening od początku, bez dostępu do internetu podczas oceniania
- **Wynik bazowy:** 31.2187

Zadanie

Burmistrz Astany chce udekorować miasto stylizowanymi logotypami IOAI. Jako statystyk postrzega wszystko — w tym logotyp — jako funkcję przestrzenną $F(x, y, \overline{W})$, gdzie $x, y \in [0, 1]$ reprezentują współrzędne na płaszczyźnie 2D, a $\overline{W}$ jest zbiorem ukrytych parametrów definiujących atrybuty stylistyczne, takie jak kolory i kąty liter.

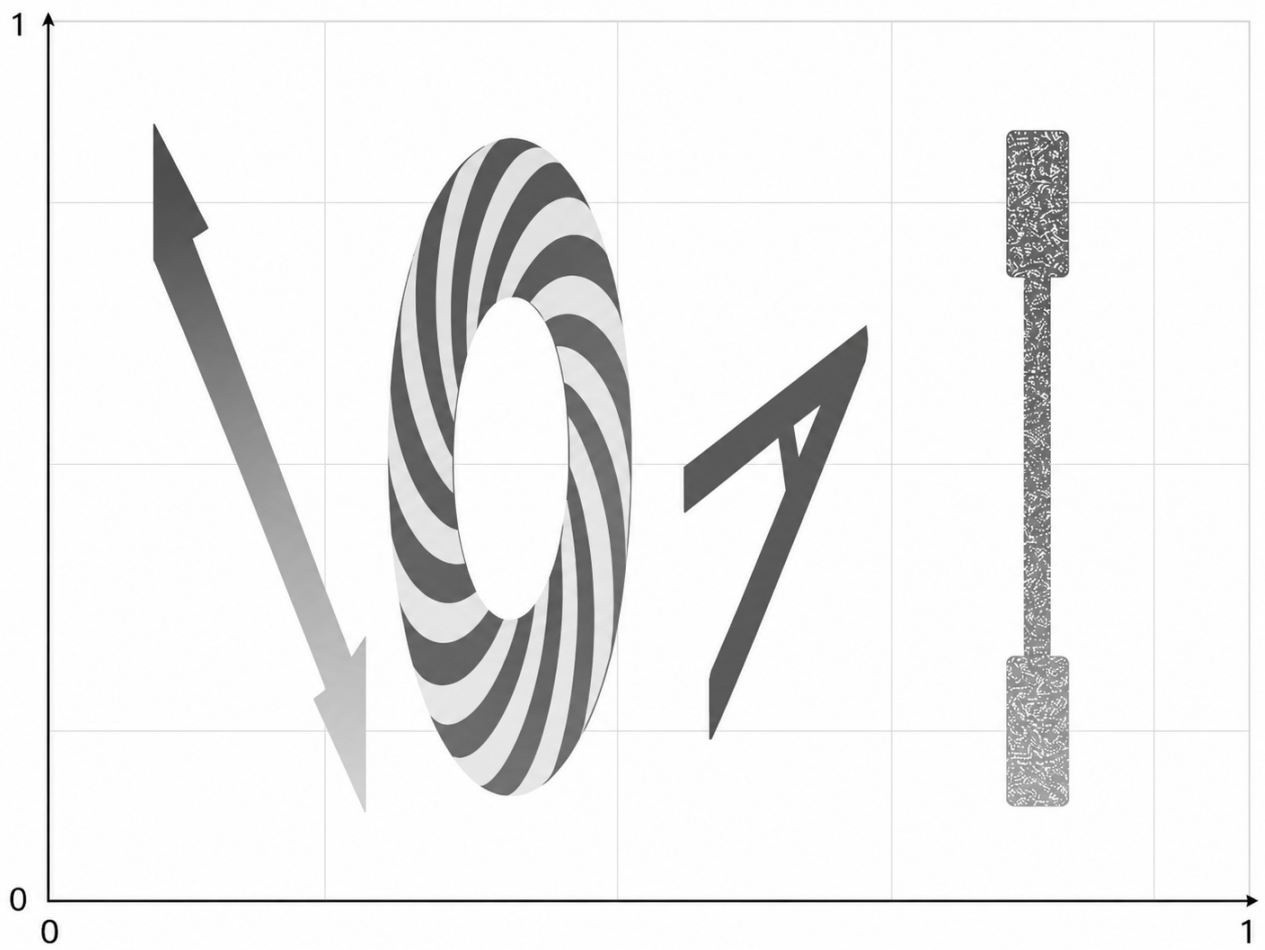
Ponieważ funkcja F jest zbyt złożona, aby wyrazić ją za pomocą jawnego równania matematycznego, Twoim zadaniem jest wytrenowanie sieci neuronowej, która będzie ją aproksymować. Dla dowolnej pary współrzędnych (x, y) sieć zwróci wartość **poła IOAI**, generując pełną wizualizację logotypu na płaszczyźnie w postaci heatmapy. Oto przykład wizualizacji funkcji F w postaci heatmapy dla pewnych konkretnych parametrów ukrytych $\overline{W}$.

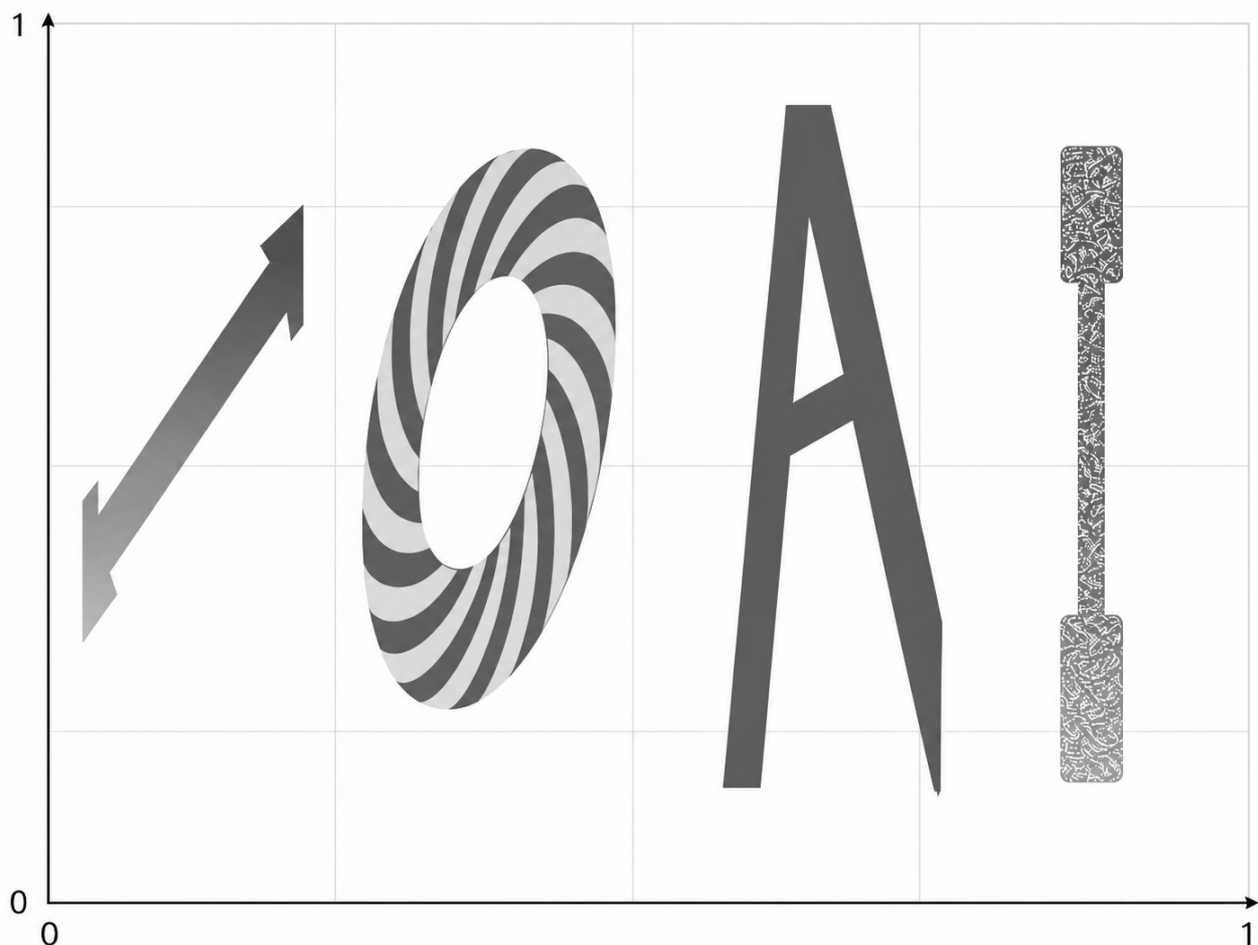


Z czego składa się pole IOAI? Z czterech liter i tła.

- Wartości wewnątrz pierwszej litery $\mathbb{I}$ są bardzo duże ($1e+10$ i większe) i mają gradient liniowy
- Wartości w literze $\circ$ tworzą wzór spiralny
- Wartość wewnątrz litery $\mathbb{A}$ zawsze wynosi -1
- Wartości wewnątrz ostatniej litery $\mathbb{I}$ powinny być wartościami losowymi z zakresu $[-2026, 2026]$, nawet gdy ten sam punkt zostanie obliczony dwukrotnie
- Poza literami wartość zawsze wynosi zero

Funkcja ma ukryte parametry $\overline{W}$, które wpływają na skalę i nachylenie liter, a także na zakres wartości wewnątrz pierwszej litery $\mathbb{I}$. Litery nie będą się jednak przecinać. Oto kilka ilustracyjnych przykładów wyglądu pola IOAI dla różnych wartości $\overline{W}$:





Co otrzymujesz:

To zadanie NIE zawiera żadnych datasetów. Zamiast tego otrzymujesz funkcję generatora skonfigurowaną za pomocą pliku konfiguracyjnego JSON znajdującego się w `data/train_config/field_config.json`.

Konfiguracja testowa jest ukryta, ale ma podobny charakter. Twoim zadaniem jest dopasowanie modelu do podanego generatora przy użyciu dowolnej ilości danych. Twoje rozkłady „treningowy” i „testowy” są generowane przez ten sam generator — po prostu nie wiesz, w których punktach (x_i, y_i) model zostanie oceniony.

Twoje zgłoszenie powinno składać się z:

- klasy modelu trenowanego zapisanej jako `custom_model.py`. Model ten powinien dziedziczyć po klasie `torch.nn.Module` i używać wyłącznie importów `torch`. Powinien zawierać klasę `CustomModel` używaną w notebooku `solution.ipynb`.
- notebooka `solution.ipynb`, który utworzy wagi `model.pt`

Ocenianie

Dla każdego obszaru minimalny wynik wynosi 0, a maksymalny wynik wynosi 1. Wynik końcowy jest średnią wyników ze wszystkich pięciu obszarów (po jednym dla każdej z czterech liter oraz dla tła), pomnożoną przez 100. Obowiązuje **kara za liczbę parametrów**:

Jeśli Twój model ma więcej niż 20260 parametrów, wynik zostaje zmniejszony o połowę.

Liczba parametrów jest mierzona za pomocą `sum(p.numel() for p in model.parameters())`. Oczekujemy, że model będzie również działał w trybie stochastycznym, przy czym dropout PyTorch `nn.Dropout` ma stanowić część modelu.

Dla obszarów standardowych

Dla każdego obszaru R (pierwsza litera $\mathbb{I}$, litera $\mathbb{O}$, litera $\mathbb{A}$, `Background`) oceniamy model na $N_R = 512$ punktach testowych (x_i, y_i) , których prawdziwe wartości to v_i , a predykcje to $\hat{v}_i$. Jako głównej metryki używamy znormalizowanego średniego błędu bezwzględnego (ang. Mean Absolute Error, MAE). MAE definiuje się następująco:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Normalizację przeprowadza się natomiast jako

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

gdzie $s_R > 0$ jest stałą skalującą.

Dla obszaru ostatniej litery $\mathbb{I}$

W tym obszarze **dropout jest włączony podczas ewaluacji**. Dla każdego punktu testowego j :

1. Uruchamiamy model $K = 10$ razy, aby otrzymać $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Jeśli którakolwiek wartość wyjściowa znajduje się poza zakresem $[-2026, 2026]$, wówczas $\text{pointScore}(j) = 0$.
3. W przeciwnym razie obliczamy odchylenie standardowe σ_j wartości wyjściowych K i przekształcamy je w wynik:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

gdzie $s_E > 0$ jest ustaloną stałą skalującą.

Wynik obszaru jest średnią ze wszystkich punktów w tym obszarze:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

gdzie $N_E = K * N_R$.

Mówiąc prościej, im większą różnorodność uzyskasz, tym większy będzie Twój wynik dla tego obszaru. **Nie wolno używać losowości w czystej postaci, w tym funkcji PyTorch `rand*` i `_uniform`; losowość powinna pochodzić z inferencji z włączonym dropoutem.**

Jak przesłać rozwiązanie

1. Otwórz `solution.ipynb` i uruchom wszystkie komórki.
2. Ulepsz model `CustomModel` w `custom_model.py`
3. Upewnij się, że ostatnia komórka zapisuje model do pliku `model.pt`.
4. Na karcie Git w JupyterLab wykonaj stage, skomentuj i wykonaj commit `solution.ipynb` oraz `custom_model.py`, a następnie wykonaj push.
5. Wróć na stronę konkursu i kliknij **Prześlij**. Komentarz do zgłoszenia powinien być taki sam jak komentarz z poprzedniego kroku.